Slide 9 – Manual Knowledge Graph Construction

"Now, in the next section, we move from ontology design to knowledge graph construction and evaluation."

"Here, I'll explain how we represented our domain as a knowledge graph, enriched it with external knowledge, integrated it into GraphDB, and finally validated the resulting knowledge graph using SPARQL queries."

Why did you create this graph first?

"Our goal was to understand the domain before implementation. Designing the conceptual graph first made it much easier to define ontology classes, object properties, and individuals consistently."

"We first identified the main ontology classes that represent the semantic categories of our domain."

بعد به شکل اشاره کن:

- **Student** (blue)
- **Professor** (green)
- **House** (purple)
- **Spell** (yellow)
- **School** (orange)
- **Subject** (pink)
- **Dark Wizard** (gray)

و بگو:

"Each color represents a different ontology class, making it easier to distinguish semantic categories."

---

2. Individuals (Entities)

بعد بگو:

"Each circle represents an individual (instance) belonging to one of these classes."

مثال بزن:

- Harry Potter → Student
- Hermione Granger → Student
- Ron Weasley → Student
- Draco Malfoy → Student
- Severus Snape → Professor
- Gryffindor → House
- Slytherin → House
- Hogwarts → School
- Lumos → Spell
- Expelliarmus → Spell
- Avada Kedavra → Spell
- Voldemort → Dark Wizard
- Potions → Subject

"The arrows represent ontology object properties that connect one individual to another."

مثال‌ها:

- belongsToHouse
- attendsSchool
- friendOf
- enemyOf

"Each object property was defined with appropriate domain and range restrictions."

مثال:

- belongsToHouse
  - Domain → Student
  - Range → House
- castsSpell
  - Domain → Wizard
  - Range → Spell
- teaches
  - Domain → Professor
  - Range → Subject
- **"These constraints allow the reasoner to validate the ontology and infer additional knowledge."**

Slide 10 :

- **Instead of manually writing SPARQL queries, we provided the LLM with our ontology schema and asked questions in plain English.**
- **For example, the question shown here is: 'Which professor teaches the Lumos spell?' The model successfully generated a valid SPARQL query based on our ontology vocabulary.**

- **This demonstrates that the LLM understood our custom classes and object properties, such as Professor, teaches, and Lumos, and translated the natural language question into executable SPARQL syntax."**

- **We guided the LLM by providing our ontology schema, including the classes, object properties, and their relationships. We also instructed the model to use our ontology vocabulary, such as *Professor*, *Spell*, and *teaches*, instead of generic terms. This domain-specific context enabled the LLM to generate SPARQL queries that were consistent with our ontology design.**

Slide 11 :

اگر استاد پرسید "Competency Questions یعنی چی؟"

بگو:

Competency questions are natural language questions that define what the ontology should be able to answer. They are commonly used to evaluate whether an ontology satisfies its intended requirements.

After designing our ontology in Protégé, we evaluated it using 22 competency questions.

For each question, an LLM generated a SPARQL query, which we executed and compared with the expected answer.

Slide 12 :

PREFIX hp: <http://www.semanticweb.org/utente/ontologies/2026/4/untitled-ontology-10#>

This line defines a namespace prefix.

The full ontology URI is quite long, so instead of writing the complete URI every time, we assign it the short prefix **hp**.

After this declaration, we can simply write **hp:Potions** or **hp:teaches** instead of writing the full URI.

SELECT ?professor

The **SELECT** statement specifies what information we want to retrieve.

Here, **?professor** is a variable.

We are asking GraphDB to return the individual that satisfies the query conditions.

WHERE

The **WHERE** clause defines the pattern that GraphDB should search for in the knowledge graph.

?professor hp:teaches hp:Potions .

Subject Predicate Object

Let's explain each part.

The whole triple

```
?professor hp:teaches hp:Potions .
```

means

Find every individual that has the **teaches** relationship with **Potions**.

or more naturally,

Find the professor who teaches Potions.

Slide 14 :

The first failed query asked, "Who are all Wizards?"

In Protégé, the HermiT reasoner automatically inferred that some individuals belonged to the Wizard class based on the ontology axioms.

However, when we imported the ontology into GraphDB, those inferred class memberships were not materialized as explicit RDF triples.

Since GraphDB mainly queries explicit triples unless reasoning is enabled and properly configured, the query could not retrieve all inferred Wizards.

This query relied on OWL existential restrictions and inferred knowledge generated by the reasoner.

The second failed query asked, "Which individuals cast some spell?"

Again, GraphDB only returned explicitly asserted relationships, so the inferred information was not available for querying.

The key lesson is that these failures were caused by reasoning limitations in GraphDB rather than mistakes in our ontology model.

"پس چرا GraphDB جواب نداد ولی Protégé جواب داد؟"

جواب کوتاه:

Protégé performs Description Logic reasoning using the HermiT reasoner, while GraphDB mainly queries explicit RDF triples. Unless reasoning is enabled and the inferred triples are materialized, GraphDB cannot retrieve inferred knowledge.

Slide 15:

After building and validating our ontology, we wanted to enrich our knowledge graph with additional information.

Instead of manually creating every new individual and relationship in Protégé, we collected additional knowledge from the Harry Potter Fandom Wiki.

We then used the Gemini Large Language Model to extract structured RDF-style triples from the textual information.

Next, we converted those triples into RDF/Turtle format, creating a file called `harry_potter_import.ttl`.

Finally, we imported both our original OWL ontology and the new Turtle file into GraphDB.

This allowed us to combine our manually modeled knowledge with automatically extracted knowledge in a single knowledge graph.

The main goal of this process was to transform unstructured natural language into machine-readable semantic relationships that could be queried using SPARQL.

رفرنسم :

2. Grounding the Development of Ontologies for Fictional Characters (Scotti et al., 2025) ⭐ ⭐ ⭐ ⭐

این مقاله درباره‌ی این است که:

برای ساخت آنتولوژی شخصیت‌های داستانی، اول باید شخصیت‌ها، نقش‌ها و روابطشان مدل شوند.

یعنی قبل از پیاده‌سازی OWL، باید بفهمی:

- شخصیت‌ها چه هستند
- چه روابطی دارند
- چه ساختاری دارند

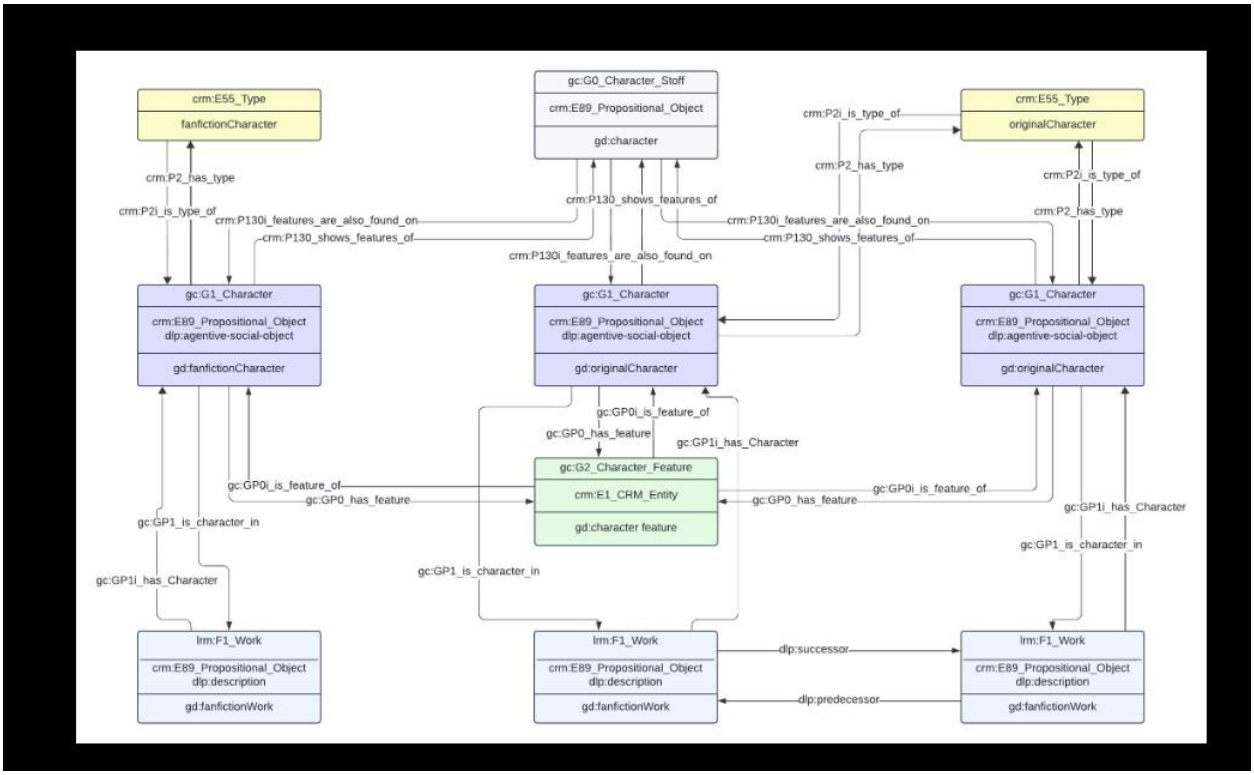
که دقیقاً همان کاری است که گراف دستی شما انجام داده.

می‌توانی بگویی:

"Our manual graph was also inspired by the ontology engineering process proposed by Scotti et al., where fictional characters and their semantic relationships are identified before formal ontology implementation."

1. GOLEM Ontology for Narrative and Fiction (Pianzola et al., 2025)

این تصویری است که داخل مقاله آمده:



معانی مفاهیم :

Ontology

A formal model that defines concepts, relationships, and rules within a specific domain.

Knowledge Graph

A graph that stores entities and their semantic relationships in a machine-readable format.

Domain

The subject area or field being modeled (e.g., the Harry Potter universe).

Class

A category or type of entity, such as Student, Professor, or Spell.

Individual (Instance)

A specific member of a class, such as Harry_Potter or Gryffindor.

Entity

Any real or fictional object represented in the knowledge graph.

Object Property

A relationship between two individuals (e.g., `belongsToHouse`, `friendOf`).

Data Property

A property that links an individual to a literal value, such as text or a number (e.g., `hasName`, `hasAge`).

RDF Triple

An **RDF Triple** is the basic building block of an RDF knowledge graph.

It represents **one semantic fact** using **three parts**:

Subject → Predicate → Object

You می‌توانی این را این‌طوری بگویی:

"Every piece of knowledge in an RDF graph is stored as a triple consisting of a subject, a predicate, and an object."

The three parts

1. Subject

The **entity** we are talking about.

It is usually a person, place, spell, house, or another resource.

Example:

Harry_Potter

---

2. Predicate

The **relationship** or property connecting the subject to the object.

Examples:

- belongsToHouse
- friendOf
- castsSpell

- attendsSchool

---

3. Object

The entity or value that completes the statement.

Example:

Gryffindor

The basic unit of RDF consisting of **Subject – Predicate – Object**.

Example:
Harry_Potter – belongsToHouse – Gryffindor

Subject

The entity that performs or owns the relationship.

Predicate

The relationship connecting two entities.

Object

The entity or value receiving the relationship.

RDF (Resource Description Framework)

A standard model for representing data as triples on the Semantic Web.

Turtle (.ttl)

A human-readable serialization format for RDF data.

OWL (Web Ontology Language)

A language for building ontologies with logical rules and reasoning support.

Protégé

An ontology editor used to design and validate OWL ontologies.

GraphDB

A graph database used to store RDF data and execute SPARQL queries.

SPARQL

The query language used to retrieve and manipulate RDF data.

Reasoner (HermiT)

A reasoning engine that checks ontology consistency and infers new knowledge.

Inference

Automatically deriving new facts from existing ontology rules.

Consistency Checking

Verifying that the ontology contains no logical contradictions.

Domain (Object Property)

Specifies **which class can be the subject** of a property.

Example:
`castsSpell` → Domain = Wizard

Range

Specifies **which class can be the object** of a property.

Example:
`castsSpell` → Range = Spell

Inverse Property

Two properties representing opposite directions of the same relationship.

Example:
`belongsToHouse ↔ hasMember`

Symmetric Property

If A is related to B, then B is automatically related to A.

Example:
`friendOf`

Disjoint Classes

Classes that cannot share the same individual.

Example:
A person cannot be both a **Student** and a **Professor**.

Existential Restriction

A rule stating that an individual **must have at least one** specified relationship.

Example:
Every Student belongsTo at least one House.

Competency Question

A natural language question used to evaluate whether the ontology can answer the intended queries.

LLM (Large Language Model)

An AI model used to extract knowledge or generate SPARQL queries.

RDF Materialization

The process of storing inferred facts as explicit RDF triples.

Explicit Triple

A fact directly stored in the RDF data.

Inferred Triple

A fact generated automatically by the reasoner rather than manually stored.